

JavaScript Fundamentals — JavaScript-এর ভিত্তি

এই Chapter-এ আমরা JavaScript-এর একদম foundation তৈরি করব। পরের chapter-গুলো বুঝতে এই অংশগুলো গভীরভাবে ও নিখুঁতভাবে বোঝা খুব important।

PART 01 — OVERVIEW, ARCHITECTURE & ORIGINS (1 – 6)

1 What is JavaScript?

English Definition: JavaScript is a high-level, interpreted/programming language mainly used to make web pages interactive and dynamic.

বাংলা ব্যাখ্যা: সহজ ভাষায়, JavaScript হলো এমন একটি programming language যার মাধ্যমে আমরা একটি website-কে শুধু static না রেখে interactive এবং dynamic করতে পারি।

- HTML দিয়ে আমরা webpage-এর **structure** তৈরি করি।
- CSS দিয়ে webpage-এর **design/style** করি।
- আর JavaScript দিয়ে webpage-এর **behavior/functionality** তৈরি করি।

🏠 REAL-LIFE EXAMPLE

ধরো তুমি একটা house বানাচ্ছে:

Technology	কাজ	Real-life analogy
HTML	Structure	বাড়ির কাঠামো
CSS	Design	রং, furniture, decoration
JavaScript	Behavior	দরজা খোলা, light on/off

যেমন একটি button: `<button>Click Me</button>` (HTML শুধু button তৈরি করবে, CSS সুন্দর করবে, কিন্তু click করলে কিছু ঘটাতে JavaScript ব্যবহার করা হয়):

HTML + JAVASCRIPT

INLINE EVENT

```
<button onclick="alert('Hello!')">Click Me</button>
```

OUTPUT

Hello! (Alert Dialog)

2 Why Do We Need JavaScript?

একটি webpage JavaScript ছাড়াও তৈরি করা যায়। কিন্তু JavaScript ছাড়া interactive functionality তৈরি করা অসম্ভব হয়ে যায়।

JAVASCRIPT দিয়ে আমরা করতে পারি:

- Button click handling & User interactions
- Form validation (Email, password check)
- Dropdown menu, Modal popups & Image slider
- Calculator, To-do application & Dynamic lists
- Dark / Light mode toggle
- API থেকে data নেওয়া (Fetch / Async)
- Search functionality & Real-time updates
- Authentication-related frontend logic & Browser storage

HTML + JAVASCRIPT

DOM MANIPULATION

```
<button onclick="changeText()">Click Me</button>
<p id="message">Hello</p>

<script>
function changeText() {
    document.getElementById("message").textContent = "Welcome!";
}
</script>
```

OUTPUT

প্রথমে: Hello
Button click করার পর: Welcome!

3 JavaScript vs Java

এটা beginner-দের একটি common confusion: **JavaScript এবং Java একই language নয়।** নাম কাছাকাছি হলেও দুটো সম্পূর্ণ আলাদা programming language।

JavaScript	Java
Mainly web development-এ ব্যবহৃত	General-purpose/backend/enterprise ইত্যাদিতে ব্যবহৃত
Browser-এ directly run করতে পারে	সাধারণত JVM-এর মাধ্যমে চলে
Dynamic language	Statically typed language
<code>.js</code> file	<code>.java</code> source file
Frontend + Backend (Node.js)	Backend + Enterprise + Android history ইত্যাদি

SYNTAX তুলনামূলক উদাহরণ:

JAVASCRIPT VS JAVA

COMPARISON

```
// JavaScript:
let name = "Ripon";
console.log(name);

// Java:
String name = "Ripon";
System.out.println(name);
```

4 JavaScript-এর ইতিহাস

JavaScript-এর শুরু হয় **1995 সালে** Netscape-এর web browser environment-এ। প্রথমদিকে এর নাম ছিল **Mocha**, পরে **LiveScript**, এবং শেষ পর্যন্ত **JavaScript** নামটি জনপ্রিয় হয়।

JavaScript-এর creator হিসেবে **Brendan Eich**-কে উল্লেখ করা হয়। তিনি Netscape-এ মাত্র ১০ দিনে JavaScript-এর প্রথম version তৈরি করেন।

TIMELINE — সহজভাবে

```
1995
↓
JavaScript introduced (Brendan Eich at Netscape)
↓
Browser scripting becomes popular
↓
ECMAScript standardization (ECMA-262)
↓
ES5 (2009 - Modern baseline)
↓
ES6 / ES2015 (Major transformation)
↓
Modern JavaScript (Yearly releases: ES2016 - Present)
```

5 What is ECMAScript?

English: ECMAScript is the standardized specification that defines how JavaScript should work.

বাংলা: ECMAScript হলো JavaScript-এর standard/specification।

সহজে মনে রাখো: JavaScript হলো language implementation, আর ECMAScript হলো তার standard/specification blueprint।

ECMAScript specification-এর সংস্করণ ক্রম: [ES1](#) → [ES2](#) → [ES3](#) → [ES5](#) → [ES6 \(ES2015\)](#) → [ES2016](#) → ... → [Modern ECMAScript](#)

6 What is ES6?

ES6 হলো JavaScript-এর একটি অত্যন্ত গুরুত্বপূর্ণ version, যেটার official নাম **ECMAScript 2015 (ES2015)**। এটি JavaScript-কে একটি আধুনিক ও অত্যন্ত শক্তিশালী ভাষায় রূপান্তর করে।

```
JAVASCRIPT (ES6) MODERN SYNTAX

let name = "Ripon";
const age = 22;

const greet = () => {
  console.log("Hello");
};
```

ES6-এ আসা প্রধান ফিচারসমূহ:

`let`, `const`, Arrow Functions, Classes, Template Literals, Destructuring, Spread Operator, Rest Parameters, Promises, Modules এবং আরও অনেক আধুনিক সিনট্যাক্স। তাই modern JavaScript শেখার ক্ষেত্রে **ES6+ খুব গুরুত্বপূর্ণ**।

7 How JavaScript Works?

যখন তুমি লিখো `console.log("Hello");`, Browser সরাসরি এই text execute করে না। Browser-এর ভিতরে একটি **JavaScript Engine** থাকে যা কোড প্রসেস করে।

SIMPLIFIED PROCESS FLOW

```
JavaScript Code
  ↓
JavaScript Engine
  ↓
Parsing / Compilation (AST & JIT Bytecode)
  ↓
Execution
  ↓
Output
```

JAVASCRIPT

ENGINE EXECUTION

```
let x = 10;
let y = 20;

console.log(x + y);
```

OUTPUT

30

8 What is a JavaScript Engine?

English: A JavaScript engine is a program that parses, compiles/interprets, and executes JavaScript code.

বাংলা: JavaScript Engine হলো এমন একটি software component যা JavaScript code-কে process এবং execute করে।

POPULAR JAVASCRIPT ENGINES

Engine	Used by
V8	Google Chrome, Chromium, Node.js, Deno
SpiderMonkey	Mozilla Firefox
JavaScriptCore (Nitro)	Apple Safari, Bun
Chakra	Older Microsoft Edge / IE

★ **Important:** Google Chrome এবং Node.js দুটোই Google-এর অত্যন্ত দ্রুতগতির **V8 Engine** ব্যবহার করে।

9 JavaScript কোথায় Run করতে পারে?

JavaScript শুধু browser-এ চলে—এটা এখন আর পুরোপুরি correct নয়। JavaScript বিভিন্ন environment-এ চলতে পারে:

- 1. **Browser:** Chrome, Firefox, Edge, Safari ইত্যাদি।
- 2. **Server: Node.js** ব্যবহার করে JavaScript দিয়ে সম্পূর্ণ server-side ও backend অ্যাপ্লিকেশন তৈরি করা যায়।
- 3. **অন্যান্য Runtime:** Deno, Bun ইত্যাদি আধুনিক হাই-পারফরম্যান্স রানটাইম।

```
JavaScript
├── Browser (DOM, Window, UI)
├── Node.js (Server, File System, OS API)
├── Deno (Secure Runtime)
└── Bun (High-speed Native Runtime)
```

10 JavaScript in Browser

Browser-এ JavaScript চালানোর সবচেয়ে simple example:

HTML DOCUMENT

INDEX.HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript</title>
</head>
<body>
  <h1>Hello JavaScript</h1>
  <script>
    console.log("Hello JavaScript!");
  </script>
</body>
</html>
```

OUTPUT

Webpage-এ: Hello JavaScript
Developer Console-এ: Hello JavaScript!

11 How to Open Browser Console?

Chrome / Edge ব্রাউজারে কনসোল খোলার উপায়:

Right Click → Inspect → Console tab
অথবা কীবোর্ড শর্টকাট: F12 (Windows) / Ctrl + Shift + I

কনসোলে সরাসরি টাইপ করতে পারো:

DEVTOOLS CONSOLE

REPL

```
console.log("Hello World");
```

OUTPUT

Hello World

PART 03 — HTML INTEGRATION, LOADING & SYNTAX RULES (12 – 22)

12 Three Ways to Add JavaScript to HTML

HTML-এর সাথে JavaScript যুক্ত করার ৩টি উপায় রয়েছে: **1. Inline**, **2. Internal**, এবং **3. External**।

12.1 INLINE JAVASCRIPT

HTML

INLINE JS

```
<button onclick="alert('Hello')">Click Me</button>
```

❌ **Problem:** বড় প্রজেক্টে Inline JavaScript maintain করা খুব কঠিন। তাই professional কোডে এটি পরিহার করা হয়।

13 Internal JavaScript

HTML ফাইলের ভেতরে `<script>` ট্যাগের মধ্যে কোড লেখা:

HTML

INTERNAL SCRIPT

```
<!DOCTYPE html>
<html>
<body>
  <h1>My Website</h1>
  <script>
    console.log("Hello JavaScript");
  </script>
</body>
</html>
```

14 External JavaScript

Professional development-এ এটিই সবচেয়ে আদর্শ ও বহুল ব্যবহৃত পদ্ধতি। কোড আলাদা `.js` ফাইলে রাখা হয়।

INDEX.HTML

EXTERNAL LINK

```
<script src="script.js"></script>
```

SCRIPT.JS

SOURCE FILE

```
console.log("Hello from JavaScript!");
```

কেন External JavaScript ভালো?

- কোড সাজানো ও organized থাকে
- সহজে maintain করা যায়
- বারবার reuse করা যায়
- HTML ক্লিন থাকে
- বড় প্রজেক্ট সহজে পরিচালনা করা যায়।

15 The `<script>` Tag

HTML-এ জাভাস্ক্রিপ্ট লোড করার জন্য `<script>` ট্যাগ ব্যবহার করা হয়। External ফাইলের জন্য `src` attribute ব্যবহৃত হয়:

HTML ELEMENT

SYNTAX

```
<script src="script.js"></script>
```

এখানে `script` হলো HTML element/tag, আর `src` হলো attribute।

16 `defer` Attribute

External script লোডের ক্ষেত্রে `defer` অত্যন্ত গুরুত্বপূর্ণ:

HTML

DEFER ATTRIBUTE

```
<script src="script.js" defer></script>
```

`defer` কী করে? Browser ব্যাকগ্রাউন্ডে HTML পার্স করার সাথে সাথে স্ক্রিপ্ট ডাউনলোড করে, কিন্তু কোড এক্সিকিউশন শুরু করে কেবল সম্পূর্ণ HTML পার্সিং সমাপ্ত হওয়ার পর।

HTML parsing + JS downloading (Parallel)

↓

HTML parsing complete

↓

JS execution

17 `async` Attribute

আরেকটি অ্যাট্রিবিউট হলো `async` :

HTML

ASYNC ATTRIBUTE

```
<script src="script.js" async></script>
```

defer:

Download ———→ HTML parsing ———→ HTML complete → JS execute

async:

HTML parsing ———→ (Parsing paused when ready)
↓
JS download → JS execute immediately

18 JavaScript Comments

Comment হলো এমন লেখা যা ইঞ্জিন এক্সিকিউট করে না, ডেভেলপারদের বোঝার সুবিধার্থে ব্যবহার করা হয়।

SINGLE-LINE & MULTI-LINE COMMENTS

JAVASCRIPT

COMMENTS

```
// This is a single-line comment
console.log("Hello");

/*
   This is a
   multi-line comment
*/
console.log("Hello");

// Real-life example: Store user's name
let name = "Ripon";
```

OUTPUT

```
Hello
Hello
```

19 JavaScript Statements

English: A statement is an instruction that tells JavaScript to perform an action.

বাংলা: Statement হলো JavaScript-কে দেওয়া একটি সুনির্দিষ্ট instruction বা আদেশ।

JAVASCRIPT

MULTIPLE STATEMENTS

```
let name = "Ripon";
let age = 22;

console.log(name);
console.log(age);
```

OUTPUT

```
Ripon
22
```

20 Semicolon `;`

JavaScript স্টেটমেন্টের শেষে সেমিকোলন ব্যবহার করা যায়। JavaScript-এ **Automatic Semicolon Insertion (ASI)** মেকানিজম থাকায় এটি অনেক ক্ষেত্রে optional হলেও প্রফেশনাল স্ট্যান্ডার্ড বজায় রাখতে সেমিকোলন ব্যবহার সুপারিশ করা হয়।

JAVASCRIPT

CONSISTENT STYLE

```
let age = 22;
console.log(age);
```

21 Expressions

Expression: An expression is code that produces a value.

যে কোনো কোড যা হিসাব শেষে একটি মান তৈরি করে তাই Expression।

JAVASCRIPT

EXPRESSIONS

```
10 + 20    // produces 30
5 * 10     // produces 50

let x = 10;
x + 5      // produces 15 (Expression)
```

22 Statement vs Expression

Expression

Statement

যেটি একটি নির্দিষ্ট value তৈরি করে

যেটি জাভাস্ক্রিপ্টকে কোনো কাজ করার instruction দেয়

যেমন: `10 + 20` (মান: ৩০)

যেমন: `let result = 10 + 20;` (পুরো লাইনটি স্টেটমেন্ট)

PART 04 — CONSOLE, VARIABLES & MEMORY SEMANTICS (23 – 35)

23 `console.log()`

জাভাস্ক্রিপ্ট শেখা ও ডিবাগিংয়ের প্রধান হাতিয়ার হলো `console.log()`। এটি কনসোলে মান প্রিন্ট করে দেখায়:

JAVASCRIPT

CONSOLE OUTPUT

```
console.log(10);
console.log("JavaScript");
console.log(true);

let name = "Ripon";
let age = 22;
console.log(name, age); // একাধিক মান কমা দিয়ে
```

OUTPUT

```
10
JavaScript
true
Ripon 22
```

24 Variables

English: A variable is a named container/reference used to store or hold a value.

বাংলা: Variable হলো এমন একটি নাম/identifier যার মাধ্যমে আমরা কোনো value ধরে রাখতে বা reference করতে পারি।

REAL-LIFE EXAMPLE

একটি লেবেলযুক্ত বাক্সের মতো: বাক্সের লেবেল `name`, আর ভেতরে রাখা মান `"Ripon"`।

JAVASCRIPT

VARIABLE DECLARATION

```
let name = "Ripon"; // name → variable, "Ripon" → value
```

25 Creating a Variable

জাভাস্ক্রিপ্টে ভেরিয়েবল ডিক্লেয়ারের ৩টি কিওয়ার্ড:

`var` (পুরাতন), `let` (আধুনিক পরিবর্তনশীল), `const` (আধুনিক অপরিবর্তনশীল)।

JAVASCRIPT

DECLARATION

```
let name = "Ripon";  
console.log(name);
```

OUTPUT

Ripon

26 `let`

`let` ব্যবহার করে এমন variable তৈরি করা যায় যার value পরে পরিবর্তন বা reassign করা সম্ভব:

JAVASCRIPT

REASSIGNMENT WITH LET

```
let age = 22;  
console.log(age);  
  
age = 23; // মান পরিবর্তন  
console.log(age);
```

OUTPUT

22
23

age (22) —(Reassigned)— age (23)

27 `const`

`const` ব্যবহার করে এমন binding তৈরি করা হয় যার মান পরবর্তীতে reassign করা যায় না:

JAVASCRIPT

CONSTANT

```
const country = "Bangladesh";  
console.log(country);  
  
// country = "India"; // TypeError: Assignment to constant variable!
```

OUTPUT

Bangladesh

28 `var`

`var` হলো জাভাস্ক্রিপ্টের পুরোনো ভেরিয়েবল ডিক্লারেশন কিওয়ার্ড। Modern কোডে এর ব্যবহার অনুৎসাহিত করা হয় কারণ এর ফাংশন স্কোপ এবং হোইস্টিং অনিচ্ছাকৃত বাগ তৈরি করতে পারে:

JAVASCRIPT

LEGACY VAR

```
var name = "Ripon";
console.log(name);
```

OUTPUT

Ripon

29 `let` vs `const` vs `var`

Feature	var	let	const
Modern code-এ ব্যবহার	কম ব্যবহৃত (Legacy)	বেশি ব্যবহৃত	সবচেয়ে বেশি ব্যবহৃত
Reassign করা যায়?	✓ হ্যাঁ	✓ হ্যাঁ	✗ না
Block scoped?	✗ না (Function scoped)	✓ হ্যাঁ	✓ হ্যাঁ
Redeclare একই স্কোপে?	✓ সম্ভব	✗ এরর দেয়	✗ এরর দেয়
Hoisting Behavior	Undefined ইনিশিয়ালাইজড	TDZ (Temporal Dead Zone)	TDZ

★ **Beginner Golden Rule:** সর্বদা `const` দিয়ে শুরু করবে। যদি পরবর্তীতে মান reassign করার প্রয়োজন পড়ে, তখন `let` ব্যবহার করবে।

30 Variable Naming Rules

ভেরিয়েবল নামকরণের কঠোর নিয়মাবলী:

VALID IDENTIFIERS:

```
let name; , let userName; , let user_name; , let age2; , let $price; , let _value;
```

INVALID IDENTIFIERS:

```
let 2age; (সংখ্যা দিয়ে শুরু করা নিষিদ্ধ!)
```

```
let user-name; (হাইফেন বিয়োগ চিহ্ন হিসেবে গণ্য হয়, চলকে ব্যবহার নিষিদ্ধ!)
```

31 JavaScript Case Sensitive

JavaScript সম্পূর্ণ **case-sensitive**! অর্থাৎ ছোট হাতের ও বড় হাতের অক্ষরের চলক সম্পূর্ণ আলাদা সত্তা:

JAVASCRIPT

CASE SENSITIVITY

```
let name = "Ripon";

console.log(name); // Ripon
// console.log(Name); // ReferenceError: Name is not defined
```

মনে রাখবে: `name` ≠ `Name` ≠ `NAME` (প্রতিটি আলাদা identifier)।

32 Naming Convention

JavaScript ইকোসিস্টেমে সর্বদা **camelCase** কনভেনশন অনুসরণ করা হয়:

GOOD (CAMELCASE):

```
firstName, lastName, userAge, totalPrice, isLoggedIn
```

AVOID:

```
firstname (পড়তে কষ্ট), FirstName (ক্লাসের জন্য সংরক্ষিত), first_name (পাইথনে ব্যবহৃত হয়, JS-এ পরিহার্য)।
```

33 Multiple Variables

JAVASCRIPT

MULTIPLE DECLARATIONS

```
let name = "Ripon";
let age = 22;
let country = "Bangladesh";

console.log(name);
console.log(age);
console.log(country);
```

OUTPUT

```
Ripon
22
Bangladesh
```

34 Variable Reassignment

JAVASCRIPT

REASSIGNMENT FLOW

```
let score = 50;
score = 80; // Updated
console.log(score);
```

OUTPUT

```
80
```

35 Constant Value

JAVASCRIPT

CONSTANT PI

```
const pi = 3.14159;
console.log(pi);

// pi = 4; // TypeError: Assignment to constant variable!
```

OUTPUT

```
3.14159
```

PART 05 — TYPES, typeof & PRACTICAL DEMONSTRATIONS (36 – 41)

36 Primitive Data Types — Introduction

জাভাস্ক্রিপ্টে মূলত ৭টি প্রিমিটিভ টাইপ ও ১টি নন-প্রিমিটিভ (Object) টাইপ রয়েছে:

```
Primitive Data Types:
String, Number, BigInt, Boolean, Undefined, Null, Symbol

Non-Primitive Category:
Object (Array, Function, Object)
```

JAVASCRIPT

BASIC TYPE DECLARATIONS

```
let name = "Ripon";           // String
let age = 22;                  // Number
let isStudent = true;         // Boolean
let result;                    // Undefined
let data = null;               // Null
```

37 `typeof`

কোনো value কোন type-এর তা রানটাইমে জানার জন্য `typeof` অপারেটর ব্যবহার করা হয়:

JAVASCRIPT

CHECKING TYPES

```
console.log(typeof "Hello"); // string
console.log(typeof 100);     // number
console.log(typeof true);    // boolean
```

OUTPUT

```
string
number
boolean
```

38 Complete `typeof` Example

JAVASCRIPT

TYPEOF SUITE

```
console.log(typeof "Ripon");    // string
console.log(typeof 22);         // number
console.log(typeof true);       // boolean
console.log(typeof undefined);  // undefined
console.log(typeof 123n);       // bigint
console.log(typeof null);       // object (Historical Quirk!)
```

OUTPUT

```
string
number
boolean
undefined
bigint
object
```

Historical Bug: `typeof null === "object"` জাভাস্ক্রিপ্টের প্রথম সংস্করণের একটি পরিচিত ঐতিহাসিক বাগ যা ব্যাকওয়ার্ড কম্প্যাটিবিলিটির জন্য অপরিবর্তিত রাখা হয়েছে।

39 Basic JavaScript Example

এখন আমরা কয়েকটা কনসেপ্ট একসাথে যুক্ত করে একটি বাস্তব উদাহরণ তৈরি করি:

JAVASCRIPT

COMBINED LOGIC

```
const name = "Shariar";
let age = 22;
const country = "Bangladesh";

console.log("Name:", name);
console.log("Age:", age);
console.log("Country:", country);
```

OUTPUT

```
Name: Shariar
Age: 22
Country: Bangladesh
```

40 Mini Real-Life Example — Student Information

JAVASCRIPT

STUDENT PROFILE

```
const studentName = "Shariar Ahamed Ripon";
const studentId = "231-15-010";
let semester = 8;
const department = "CSE";

console.log("Student Name:", studentName);
console.log("Student ID:", studentId);
console.log("Semester:", semester);
console.log("Department:", department);
```

OUTPUT

```
Student Name: Shariar Ahamed Ripon
Student ID: 231-15-010
Semester: 8
Department: CSE
```

প্রোগ্রামিংয়ের মূল দর্শন: Data → Store in Variables → Process Logic → Output to User.

41 JavaScript-এর Basic Mental Model

EXECUTION MENTAL MODEL

```
graph TD
    A[JavaScript] --> B[Write Code (.js)]
    B --> C[JavaScript Engine (V8)]
    C --> D[Process / Execute (JIT / Call Stack)]
    D --> E[Result]
```

WEB ARCHITECTURE TRINITY

```
HTML (Structure) + CSS (Design) + JavaScript (Behavior & Logic)
↓
Web Application
```

Chapter 1 — Quick Revision

Topic	মূল কথা
JavaScript	Webpage-কে dynamic ও interactive করতে ব্যবহৃত language
ECMAScript	JavaScript-এর standardized specification
ES6	ECMAScript 2015; modern JS-এর গুরুত্বপূর্ণ milestone
JavaScript Engine	JavaScript code execute করে
V8	Chrome ও Node.js-এর JavaScript engine
Browser	JavaScript run করার একটি ক্লায়েন্ট environment
Node.js	Browser-এর বাইরে JavaScript runtime
<code><script></code>	HTML-এর সাথে JS যুক্ত করার element
<code>defer</code>	HTML parsing শেষের পর script execute করতে সাহায্য করে
Comment	Code explanation-এর জন্য (ইঞ্জিন এড়িয়ে চলে)
Statement	JavaScript-এর instruction
Expression	Value produce করে
Variable	Value/reference রাখার জন্য named binding
<code>let</code>	Reassign করা যায়
<code>const</code>	Reassign করা যায় না
<code>var</code>	পুরোনো variable declaration mechanism
<code>typeof</code>	Value-এর type জানতে সাহায্য করে
Case Sensitive	<code>name</code> এবং <code>Name</code> আলাদা চলক

Chapter 1 Practice Exercises

Practice 1 — Personal Info:

তোমার Name, Age, University, Department এবং Country চলকে সংরক্ষণ করে `console.log()` দিয়ে প্রিন্ট করো।

Practice 2 — Type Detection:

নিচের ভ্যালুগুলোর টাইপ `typeof` দিয়ে বের করো:

```
"Hello"
100
true
undefined
null
123n
```

Practice 3 — Reassignment:

একটি `let` ডায়রিয়েবল `score = 50` তৈরি করো, তারপর সেটাকে `80` -তে পরিবর্তন করে কনসোলে প্রিন্ট করো।

Practice 4 — Constant Test:

একটি `const` variable তৈরি করে পরে পরিবর্তন করার চেষ্টা করো এবং browser console-এ কী error আসে তা পর্যবেক্ষণ করো।

Practice 5 — Mini Project (Student Information System):

একটি পূর্ণাঙ্গ ছাত্র বিবরণী তৈরি করো: Name, ID, Department, Semester, CGPA, University। সব ডেটা ডায়রিয়েবলে রেখে কনসোলে ফরম্যাটেড আকারে প্রিন্ট করো।

★ Chapter 1-এর সবচেয়ে গুরুত্বপূর্ণ takeaway

এই ফ্লো-চি মাথায় গভীরভাবে গেঁথে নাও:

JavaScript → Variables → Data → Expressions → Statements → JavaScript Engine (V8) → Execution → Output

Chapter 2-এ আমরা JavaScript-এর Variables, Data Types & Type System আরও গভীরভাবে ধরব — String, Number, Boolean, Undefined, Null, BigInt, Symbol, Object, typeof, primitive vs non-primitive, memory/reference ধারণা, এবং type conversion/coercion ধাপে ধাপে code + output সহ।